

고급자료구조

VITE 설명서

20203067 서현우

1. MAKEFILE

윈도우의 경우, 리눅스와 맥의 경우를 합하여 구분해 만들었습니다.

1-1.윈도우의 경우 : 같은 디렉터리안에 pdccurses관련 파일이 있다고 가정(제출한 파일에 포함.)하여 코드를 작성하였습니다.

```
ifeq ($(OS),Windows_NT)
CC = gcc
CFLAGS = -Wall -Wextra -g -I.
LDFLAGS = -L. -lpdcurses -lm
LIBS = -luser32 -lgdi32 -lpdcurses
OBJECTS = main.o

TARGET = vite

all: $(TARGET)

$(TARGET): $(OBJECTS)
$(CC) $(OBJECTS) -o $(TARGET) $(LDFLAGS) $(LIBS)

main.o: main.c
$(CC) $(CFLAGS) -c main.c

clean:
rm -f *.o $(TARGET)

.PHONY: all clean
```

2023_DSA_20203067				2023_DSA
정렬				보기
이름	수정한 날짜	유형	크기	
curses.h	2008-09-09 오전 12:09	DevCpp.h	49KB	
main.c	2023-12-18 오후 1:50	C 원본 파일	31KB	
Makefile	2023-12-13 오전 3:52	파일	1KB	
panel.h	2008-07-13 오후 12:08	DevCpp.h	2KB	
pdccurses.dll	2008-09-09 오전 12:02	응용 프로그램 확장	112KB	
pdccurses.lib	2008-09-09 오전 12:02	Object File Library	66KB	

vscode에서 mingw32-make를 실행하면 정상적으로 파일이 생성됩니다.

```
PS C:\Users\sho52\OneDrive\바탕 화면\2023_DSA_20203067> mingw32-make
gcc -Wall -Wextra -g -I. -c main.c
gcc main.o -o vite -L. -lpdcurses -lm -luser32 -lgdi32 -lpdcurses
```

1-2.리눅스 및 맥 : 기본 제공되는 curses가 있으므로 해당 curses를 -lcurses하는 옵션으로 메이크파일을 작성하였습니다.

```
else
CC = gcc
CFLAGS = -Wall -Wextra -std=c99
LIBS = -lm -lcurses

TARGET = vite

all: $(TARGET)

$(TARGET): main.o
    $(CC) $(CFLAGS) -o $(TARGET) main.o $(LIBS)

main.o: main.c
    $(CC) $(CFLAGS) -c main.c

clean:
    rm -f $(TARGET) *.o
endif
```

2. 사용한 헤더파일

```
#include <stdio.h>
#include <stdlib.h>
#include <curses.h>
#include <string.h>
#include <math.h>
```

#include <curses.h>를 주력으로 사용하여 텍스트기반의 인터페이스를 구축하였습니다.

3. 운영 체제별 전처리

운영체제별 전처리를 사용하여 각 체제별로 서로다른기능이 있다면 그에 맞도록 수행하게 정의 하였습니다.

```
#elif _WIN32
// 윈도우즈에서 Enter
#define enter 13
#define backspace 8
#define del 127
```

윈도우-윈도우에서의 엔터는 아스키13, 백스페이스는 8, delete는 127번이어서 해당 값에 맞게 정의하였습니다

```
#ifdef __linux__
// 리눅스에서 Enter 키의 ASCII 값은 10
#define enter 10
#define del 127
#define backspace 8
```

리눅스-리눅스에서의 엔터는 10번, 딜리트는 127번 백스페이스는 8번의 값이 인식되므로 해당값에 맞게 정의하였습니다.

```
#elif __APPLE__
// 맥에서 Enter 키의 ASCII 값은 10
#define enter 10
#define del 127
#define backspace 8
#define KEY_HOME 8
#define KEY_END 5
#define KEY_PPAGE 21
#define KEY_NPAGE 4
Ctrl + U: ASCII 값 21
Ctrl + D: ASCII 값 4
Ctrl + H: ASCII 값 8
Ctrl + E: ASCII 값 5
#endif
```

MAC-맥에서의 엔터는 10번, 딜리트는 127번, 백스페이스는 없지만 혹시몰라 이때까지 같은 값을 썼던 8번으로 하였고

여기서 문제가 된 주석을 달지 못한 설명인 각 특수키들(홈,엔드,업,다운)을

page-up - ctrl+u(up)의 의미

page-down - ctrl+d(down)의 의미

home - ctrl+h(home)의 의미

end - ctrl+e(end)의 의미 에 맞도록 각아스키값에 정의 하였습니다.

4. 구조체

더블 링크드리스트를 만들어 각 행을 나타내게 하였고 해당 행의 문자열 포인터를 저장하였습니다. 또한 문자열의 길이를 저장하는 strsize를 사용하였고

각 구조체는 up과 down으로 일반적인 행의 구조를 머릿속에서 상상하기 편하도록 이름 붙이고 연결하였습니다.

```
typedef struct DNode {
    int strsize;
    char* str;
    struct DNode* up;
    struct DNode* down;
} node;
```

5. 사용한 전역변수

```
int x = 0, y = 0; // 좌표값
int pressQ = 1; // ctrl+q가 눌렸는지 확인하는 변수
int pressS = 2; // ctrl+s가 눌렸는지 체크 // 2는 아무것도 안한 제일 초기상태

// UI용 전역변수
int linenum = 0; // 현재 몇라인인지 알려주는 전역변수
int conRows, conCols; // 콘솔의 행열
char* filename = NULL; // 파일이름
char* ft = NULL; // 현재라인/전체라인 no ft | y / totalnum;
int frontLength; // 파일이름포함한 길이 + 라인넘버포함한길이
int backLength; // 뒤에 ft라인 길이
int totalnum = 0;
```

int x,y - 현재 화면의 좌표값을 나타냅니다. (콘솔의 크기이상 넘어가지 않습니다.)

int pressQ - ctrl+q가 입력되었는지 체크합니다.

int pressS - 마찬가지로 ctrl_s 가 눌렸는지 체크합니다.

int linenum - 현재 커서가 파일의 몇 번째 행에 위치하는지 알려줍니다.

int conRows, conCols - 현재 콘솔의 행, 열값을 나타냅니다.

char*filename - 현재 사용중인 파일의 이름을 알려줍니다.

char*ft - 현재 위치를 알려주는 ui용 변수였는데 다른값으로 대체되었습니다.(사용x)

int frontLength - UI셋을 위한 앞쪽 글자의 길이입니다.(있는데 사용하지 않습니다.)

int backLength - UI셋을 위한 뒤쪽 글자의 길이입니다.(있는데 사용하지 않습니다.)

int totalnum - 파일의 총 라인 넘버입니다.

```
node* head = NULL; // 맨 위의 노드를 가리키는 포인터
node* curNode = NULL; // 현재 위치한 노드를 가리키는 포인터
```

head - 파일의 맨 윗 행, 맨위에있는 노드를 가리키는 포인터 노드입니다.

curNode - 현재 작업중인 위치를 가리키는 포인터입니다.

6. 핵심 함수의 설명

void setUI() - 아래 두줄의 UI 세팅을 담당하는 함수입니다. digits라는 변수를 사용하여 totalnum과 linenum이 몇글자를 차지하는지 계산 한 후 콘솔의 길이에 맞게 고백을 출력하여 UI글자들을 출력합니다.

void setWave() - UI의 물결표를 출력하는 함수입니다.

void setFirstUI() - 저장된 파일을 오픈하지않았다면 UI의 기본 화면을 출력합니다.

```
void insertAtTail(node* tmp)
```

- 입력된 노드를 맨 끝에 입력하는 함수입니다. (자료구조수업 참고하여 만들었습니다.)

```
void inputStr(node* tmp, int ch)
```

- 노드의 배열 맨 끝에 char 한글자를 넣는 함수였으나 현재는 str이 비어있는 노드에 제일 처음 글자를 인풋할때만 사용합니다.

```
void changeStr(int ch)
```

- inputStr을 만들고 change를 만들고 보니 change함수로 문자열 맨 뒤에도 삽입이 가능하고 문자열의 전체적이 수정이 가능하게 되어서 두 번째 이상 입력되는 문자들을 모두 change함수로 입력을 받습니다.

-

```
curNode->str = (char*)realloc(curNode->str, (curNode->strsize + 1) * sizeof(char));
```

input과 change 모두 미리 메모리가 잡혀있던 str의 사이즈를 realloc하여 한 글자마다 한 칸씩 크기를 늘려 작업을 수행하였습니다. +1의 이유는 문자열의 맨 마지막위치에 '\0' 을 입력하기 위함입니다.

```
void rePrintConsol()
```

- 화면의 모습을 재출력해주는 함수입니다. 즉 현재 커서의 위치에 따라 출력해야하는 화면이 변하거나 할 때 사용하는 함수입니다.

초기에는 해당 함수를 구현하지않고 변화가 일어나는 라인만 재구성하는 쪽으로 생각을 하고 구현을 하고자 하였는데 (비효율성과 깜빡임의 걱정) 막상 코드를 구현해보니 새롭게 변화가 일어나는 파트만 재구성하려니 고려해야할 사항이 매우 많았고 (ex_키보드이동함수) 막상 현재화면을 reprint해보니 화면 깜빡임도 그렇게 생기지않아서 해당함수를 과제 후반부에 많이 사용하게 되었습니다.(reprintEnt, reprintDel을 만들고 보니 해당함수들이 공통되고, 여러 곳에서 사용할 수 있을 것 같아 rePrintConsol이라는 함수로 새로만들게 된 경우입니다.)

```
void delCurNode()
```

- backspace나 delete가 x좌표의 맨 앞(열의 맨앞 자리)에서 수행이 될 경우 현재위치한 노드를 삭제하는 함수입니다.

```
void deleteChar()
```

- char삭제의 경우를 담당하는 함수입니다. 해당 함수도 realloc를 사용하여 커서의 위치에따라 문자열을 삭제하였으며 삭제가 일어날 경우 문자열의 길이를 -1씩 realloc하고 그에맞는 배열의 문자이동을 구현하였습니다.

```
void insertWant(node* tmp)
```

- 새로만들어지는 노드(ex_행의 가운데에서 엔터가 입력되는 경우가 있는 경우 해당 노드가 어느위치에 들어가야하는지 linenum의 위치에 따라 노드를 새로 삽입합니다.


```
void curMove(int ch)
```

- 화살표움직임에 따라 커서의 이동을 구현한 함수입니다.

가장 쉬운 함수일것이라 생각했으나 그렇지않았고 고려해야할 요소가 너무 많아서 상당히 까다로웠던 함수입니다. (ex_화면이 넘어가는 경우의 화면출력을 일일이 생각하고 코드를 구현 하여서 굉장히 더러운 코드가 탄생하였습니다-후에 rePrint함수를 만들었지만 만들어 놓은 코드가 아까워 유지하기로 하였습니다.)

```
void specialKey(int ch)
```

- 스페셜키는 커서무브 때 비효율적으로 화면이동을 구성한 것 같아 rePrint라는 함수를 활용하여 그나마 효율적으로 화면이동을 구현하였고, 이외의 커서이동은 일반적인 home, end, page_up, page_down과 비슷하게 구현을 하였습니다.

7. MAIN

메인문의 주요 내용을 설명하겠습니다.

```
getmaxyx(stdscr, conRows, conCols);
```

- 현재 콘솔의 행, 열을 알아와서 conRows와 conCols에 저장합니다.

file open 파트

```
if (argc > 1) {
    FILE* file = fopen(argv[1], "r");
    if (file == NULL) {
        printw("Error: File could not be opened.");
        return 1;
    }
    node* current = NULL;
    char *buffer=NULL;
    size_t len = 0;
    filename = argv[1];
    while (getline(&buffer, &len, file) != -1) {
        if (buffer[strlen(buffer) - 1] == '\n') { // 개행 문자 제거
            buffer[strlen(buffer) - 1] = '\0';
        }

        current = getN();
        insertAtTail(current);
        current->strsize = strlen(buffer);
        current->str = strdup(buffer); // strdup 함수 사용
    }
}
```

getline함수를 사용하여 파일안에있는 문자들을 한 행씩 읽어옵니다. 각 행은 노드에 삽입이 되고 개행의 작업은 reprintConsol에서 수행이 되므로 개행문자를 삭제, 해당위치에 널 문자를 삽입합니다. 해당 작업은 10MB의 경우 빠른 컴퓨터에서는 30~40초, 제 노트북에서는 약 1~2분정도가 소요되었습니다.

```

if (argc < 2) { //argc<2 : 파일오픈x
    if (filename == NULL) {
        filename = "Noname"; //나중에
    }
    setFirstUI(); //파일이름이 없는 시
}

```

만일 파일오픈을 하지않는다면 기본 VITE를 오픈하고 제일처음의 UI를 보여줍니다.

본격적인 입력 파트

`int ch;` - 입력되는 값들을 아스키값으로 인식하여 받아오는 (int ch)를 선언합니다.

`letter = value(ch);` - 입력된 값이 문자인지, 특수키인지 구분합니다.

`while (1) {` - 각 입력값들을 구분하여 그에 맞는 동작을 수행합니다. (ex_글자입력, 엔터입력, 백스페이스입력, 화살표 및 특수키 입력 등)

VITE의 종료

```

else if (letter == 0) { //문자가아닌 특수키의 경우.
    // ctrl+q면 종료 나중에 두번눌러야 종료되게실행
    if (ch == 17) {
        move(conRows - 1, 0);
        clrtoeol();
        pressQ = 1;
        if (pressS == 0 && pressQ == 1) {
            mvprintw(conRows - 1, 0, "*Not Saved* press one more to quit");
            ch = getch();
        }
        else if (pressS == 1 && pressQ == 1) {
            mvprintw(conRows - 1, 0, "*File Saved* press one more to quit");
        }
        //refresh();

        if (ch == 17) {
            break;
        }
        else { pressQ = 0; setUI(); move(y, x); }
    }
}

```

저장 되지 않은상태에서 Ctrl+q가 두 번 입력된다면 while(1)을 빠져나가 VITE가 종료됩니다.
저장이 되었다면 ctrl+q가 한번만 입력되어도 종료됩니다.

file save 파트

```
if (ch == 19) { //ctrl+s
    FILE *saveFile;
    if (strcmp(filename, "Noname") == 0) {
        //getch()로 받고 리얼록하고파일네임저장하고 등등 setUI도 새로함
        move(conRows-1,0);
        clrtoeol(); // 현재 행을 지움
        mvprintw(conRows-1,0,"Enter file name : ");
        int saveName;
        int saveIndex=0;
        char *newFilename=NULL;
        while(1){
            saveName=getch();
            if(saveName==enter){
                move(y,x);
                break;
            }
            printw("%c",saveName);
            newFilename = (char *)realloc(newFilename, (saveIndex + 2) * sizeof(char));
            newFilename[saveIndex] = saveName;
            newFilename[++saveIndex] = '\0';
        }
        filename = newFilename; // 저장할 파일 이름
    }
}
```

```
saveFile = fopen(filename, "w");
if (saveFile == NULL) {
    printw("Error: File could not be opened.");
    return 1;
}
node*cur=head;
while(cur!=NULL){
    fputs(cur->str, saveFile);
    cur=cur->down;
    fprintf(saveFile, "\n");
}
fclose(saveFile);
//savefile(); 구현하자
pressS = 1;
pressQ = 1;
```

파일의 이름을 지정받고 새로운 파일이름명으로 UI도 다시 셋을 해줍니다. 또한 파일을 저장해야하므로 fopen을 사용. filename으로 파일을 오픈 한 후 fputs로 한 행 씩 문자열을 저장하여 줍니다. 한 행을 저장하였다면 개행을 직접입력하여주고 노드가 끝날 때 까지 반복합니다.

이외에는 여러 특수키(엔터, 딜리트, 화살표키, 스페셜키 등등의 작업을 수행합니다.)

MAC에서는 특수키를

page-up - ctrl+u(up)의 의미

page-down - ctrl+d(down)의 의미

home - ctrl+h(home)의 의미

end - ctrl+e(end)의 의미 에 맞도록 각아스키값에 정의 하였습니다.

이상입니다. 감사합니다.